웹의 세계 2장

브라우저는 무슨 일을 하는가

주소창 뒤에서 벌어지는 일, 렌더링 엔진, 개발자도구 제대로 쓰기

한신대학교 AI·SW계열 · 웹프로그래밍

학습사이트 hs-web.dreamitbiz.com

학습 목표

1. 브라우저의 구성 요소와 각각의 역할을 설명할 수 있다.
2. 렌더링 엔진과 자바스크립트 엔진의 차이를 구분할 수 있다.
3. 개발자도구의 주요 탭을 목적에 맞게 골라 쓸 수 있다.
4. 캐시가 무엇이고 왜 "새로고침해도 그대로"인지 설명할 수 있다.
5. 브라우저 호환성 문제를 확인하고 대응하는 방법을 안다.

핵심 키워드

브라우저 엔진 · 렌더링 엔진 · Blink · WebKit · Gecko · V8 · 개발자도구 · 캐시 · 호환성 · caniuse

학습사이트의 본문에서 이 낱말들을 찾아 읽으면 이 시간의 뼈대가 잡힙니다

웹의 세계 2장 · 1

브라우저의 구성

브라우저는 생각보다 복잡한 프로그램입니다.

브라우저의 구성

구성 요소	하는 일
사용자 인터페이스	주소창, 뒤로 가기, 북마크 — 화면의 테두리 부분
렌더링 엔진	HTML·CSS를 해석해 화면을 그린다
자바스크립트 엔진	JS 코드를 해석하고 실행한다
네트워크	HTTP 요청·응답 처리
자료 저장소	쿠키, localStorage, IndexedDB
UI 백엔드	기본 버튼·입력칸 등 OS의 위젯 그리기

운영체제 다음으로 복잡한 소프트웨어라고들 말합니다.

1.1 엔진들

브라우저	렌더링 엔진	JS 엔진
Chrome · Edge · Whale · Opera	Blink	V8
Safari	WebKit	JavaScriptCore
Firefox	Gecko	SpiderMonkey

1.1 엔진들 (1/2)

1. 엔진이 몇 종류뿐이라는 것이 중요합니다.
2. 브라우저는 수십 가지지만 엔진은 사실상 셋뿐이고, 그중 Blink 계열이 압도적입니다.
3. 그래서 "크롬에서 되면 대부분 된다"가 성립하지만, Safari는 따로 확인하는 편이 안전합니다.

1.1 엔진들 (2/2)

1. iOS에서는 모든 브라우저가 WebKit입니다.
2. 아이폰의 크롬도 겉모습만 크롬이지 안은 사파리 엔진입니다(애플 정책).
3. 그래서 "아이폰 크롬에서만 이상해요"는 사실 "사파리에서 이상해요"인 경우가 많습니다.

웹의 세계 2장 · 2

화면을 그리는 과정

화면을 그리는 과정 (1/2)

1. **HTML 파싱**
   글자를 읽어 DOM 나무를 만듭니다.
2. **CSS 파싱**
   스타일 규칙을 읽어 CSSOM을 만듭니다.
3. **렌더 트리**
   둘을 합쳐 "실제로 보이는 것"만 남깁니다. display: none 은 여기서 빠집니다.

화면을 그리는 과정 (2/2)

1. **레이아웃(리플로우)**
   각 요소가 어디에 얼마만 한 크기로 놓일지 계산합니다.
2. **페인트**
   색·테두리·그림자를 픽셀로 칠합니다.
3. **합성**
   겹쳐진 레이어를 합쳐 최종 화면을 만듭니다.

script는 HTML 파싱을 막습니다

```
<head>
  <script src="big.js"></script>   <!-- 이걸 다 받고 실행할 때까지 화면이 안 그려짐 -->
</head>

<!-- 해결 1: 몸통 끝으로 -->
<body>
  ...
  <script src="app.js"></script>
</body>

<!-- 해결 2: defer — 병렬로 받되 DOM 완성 후 순서대로 실행 (권장) -->
<script src="app.js" defer></script>

<!-- 해결 3: async — 병렬로 받고 받는 대로 실행 (순서 보장 안 됨) -->
<script src="analytics.js" async></script>
```

2.1 파싱이 멈추는 순간

1. 이것이 "화면이 하얗게 있다가 갑자기 뜨는" 현상의 흔한 원인입니다.

2. CSS도 렌더링을 막지만(스타일 없이 그렸다가 다시 그리면 화면이 번쩍이므로 의도된 것), JS만큼은 아닙니다.

웹의 세계 2장 · 3

개발자도구 — 이 학기의 주 무기

F12(맥은 Cmd + Option + I)로 엽니다.

개발자도구 — 이 학기의 주 무기

탭	언제 여는가	주로 보는 것
Elements	"이 여백은 어디서 온 거지?"	DOM 구조, 적용된 CSS, 취소선 그어진 규칙
Console	"왜 안 되지?" — 가장 먼저	빨간 오류, console.log 출력
Network	"이미지가 왜 안 뜨지?"	상태 코드(404·500), 파일 크기, 로딩 시간
Application	"저장한 값이 어디 있지?"	localStorage, 쿠키, 캐시
Lighthouse	"내 사이트 품질이 어떻지?"	성능·접근성·SEO 점수와 개선 제안
기기 도구 모음	"휴대폰에서 어떻게 보이지?"	화면 폭별 확인

남의 사이트를 뜯어보는 도구이자, 내 코드가 왜 안 되는지 알려 주는 도구입니다.

반드시 익혀야 할 세 가지

정리

1. **요소 선택 화살표**
   (왼쪽 위) — 누르고 화면의 아무 곳이나 클릭하면 그 요소가 바로 잡힙니다.

2. **Styles 칸에서 값을 바로 고칠 수 있습니다.**
   색을 바꿔 보고 마음에 들면 그 값을 코드에 옮기세요. 새로고침하면 원래대로 돌아옵니다.

3. **취소선이 그어진 규칙**
   을 찾으세요. 그게 진 규칙입니다. 왜 졌는지(특정도인지 순서인지) 바로 위에 이긴 규칙이 보입니다.

3.1 Elements 탭 제대로 쓰기

1. 개발자도구로 남의 사이트를 바꿔도 그 사이트는 전혀 바뀌지 않습니다.
2. 내 컴퓨터에 내려온 사본을 잠깐 고친 것뿐입니다.
3. 마음 놓고 뜯어보세요 — 잘 만든 사이트를 뜯어보는 것이 가장 좋은 공부입니다.

값을 확인하는 방법들

```
console.log('값:', value);
console.log({ name, age });      // 변수 이름과 값을 함께 — 아주 유용
console.table(arrayOfObjects);   // 배열을 표로
console.warn('주의');
console.error('오류');

// 콘솔에서 직접 실행해 볼 수도 있습니다
document.querySelector('h1')     // 요소를 바로 확인
$0                               // Elements에서 선택한 요소
$$('li')                         // querySelectorAll의 축약
```

"왜 안 되지?" 할 때의 순서

정리

1. **Console을 먼저 엽니다.**
   빨간 글씨가 있으면 파일명과 줄 번호가 적혀 있습니다.
2. **의심되는 곳 앞뒤에 console.log 를 찍습니다.**
   어디까지 실행됐는지, 값이 무엇인지.
3. **undefined 나 null 이 나오면**
   그 앞 단계에서 이미 실패한 것입니다. 위로 올라가며 찾습니다.
4. **30분 이상 혼자 막히면 물어보세요.**
   그것은 인내가 아니라 낭비입니다.

웹의 세계 2장 · 4

캐시 — "새로고침해도 그대로"의 정체

브라우저는 한 번 받은 파일을 저장해 두었다가 다음에 재사용합니다.

캐시 — "새로고침해도 그대로"의 정체

1. 그래야 페이지가 빨라지기 때문입니다.
2. 그런데 이것이 개발 중에는 "고쳤는데 반영이 안 돼요" 로 나타납니다.

캐시 — "새로고침해도 그대로"의 정체

방법	단축키	효과
일반 새로고침	F5 · Cmd+R	캐시를 쓸 수 있으면 씀
강력 새로고침	Ctrl+Shift+R · Cmd+Shift+R	캐시를 무시하고 다시 받음
캐시 비우고 새로고침	개발자도구 열고 새로고침 버튼 길게 누르기	완전히 비움
개발 중 캐시 끄기	Network 탭 → Disable cache 체크	도구가 열려 있는 동안 캐시 안 씀

캐시 — "새로고침해도 그대로"의 정체

1. 배포한 사이트가 옛 내용을 보여 준다면 캐시일 가능성이 높습니다.
2. CDN을 쓰는 경우 원본 파일을 지웠는데도 최대 몇 시간 동안 옛 파일이 나가는 일이 실제로 있습니다.
3. 삭제 직후에 404가 안 뜬다고 배포 실패로 오판하지 마세요.

웹의 세계 2장 · 5

브라우저 호환성

새 CSS 속성이나 JS 기능은 브라우저마다 지원 시점이 다릅니다.

지원 여부를 코드에서 확인하기

```
/* CSS — 지원하는 브라우저만 이 블록을 읽습니다 */
@supports (display: grid) {
  .layout { display: grid; }
}

// JavaScript
if ('IntersectionObserver' in window) {
  // 최신 방식
} else {
  // 대체 방식
}
```

> **caniuse.com 에서 기능 이름을 검색하면 어느 브라우저의 어느 버전부터 되는지 표로 나옵니다.**

실무의 현실적인 기준

정리

1. **최신 크롬·사파리·파이어폭스·엣지**
   를 기준으로 만듭니다. 이들은 자동 업데이트되므로 대부분 최신입니다.
2. **Safari를 따로 확인**
   합니다. 신기능 지원이 가장 늦고, iOS 사용자가 많습니다.
3. **인터넷 익스플로러는 지원이 끝났습니다**
   (2022년). 더 이상 고려하지 않습니다.
4. **우아한 저하(graceful degradation)**
   최신 기능이 없어도 내용은 읽을 수 있게 만듭니다. 예쁘지 않을 뿐 못 쓰지는 않게.

웹의 세계 2장 · 6

브라우저를 잘 쓰는 습관

브라우저를 잘 쓰는 습관

1. **개발자도구를 항상 열어 둡니다.**
   콘솔에 빨간 글씨가 뜨는 순간 바로 알아챌 수 있습니다.
2. **Network 탭의 Disable cache를 켜 둡니다.**
   개발 중 캐시 때문에 헤매는 시간이 사라집니다.
3. **기기 도구 모음으로 수시로 확인합니다.**
   마지막에 몰아서 보면 다 뜯어고쳐야 합니다.
4. **마음에 드는 사이트를 뜯어봅니다.**
   "이건 어떻게 만들었지?"의 답이 Elements 탭에 있습니다.
5. **시크릿 창으로도 확인합니다.**
   로그인 상태나 확장 프로그램의 영향을 배제할 수 있습니다.

웹의 세계 2장 마무리

2장 마무리

브라우저는 무슨 일을 하는가 — 본문과 도해는 학습사이트에서 다시 읽습니다

- ✓ 학습사이트 장 페이지 끝의 「이해했어요 · 어려워요」 표시
- ✓ 장 끝의 확인 퀴즈로 스스로 점검
- ✓ 함께 볼 주차: 1주차 · 11주차
- ✓ 막히면 「질문있어요」 — 질문은 반 전체가 같은 자리에서 봅니다